Current	Result	Size	Time	Cycles	GPU	SM Frequency	Process	Attributes
1673 - circuit_kernel_batch_plain_stack	(977, 1, 1)x(128, 1, 1)	6.31 MB	9.203,982	0	NVIDIA GeForce RTX 2060 SUPER	1.47 GHz	[266177] python3.11	

Summary | Details | Source | Context | Comments | Raw | Session

Compare | Tools | View | Export

GPU Speed Of Light Throughput

GPU Throughput Chart

High-level overview of the throughput for compute and memory resources of the GPU. For each unit, the throughput reports the achieved percentage of utilization with respect to the theoretical maximum. Breakdowns show the throughput for each individual sub-metric of Compute and Memory to clearly identify the highest contributor. High-level overview of the utilization for compute and memory resources of the GPU presented as a roofline chart.

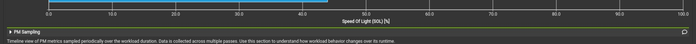
Compute (SM) [%]	67.13	Duration [ms]	6.31
Memory Throughput [%]	43.94	Elapsed Cycles [cycle]	9909802
L1/TEX Cache Throughput [%]	49.33	SM Active Cycles [cycle]	9121512
L2 Cache Throughput [%]	11.00	SM Frequency [GHz]	1.47
DRAM Throughput [%]	9.79	DRAM Frequency [GHz]	6.77

Latency Issue This workload exhibits low compute throughput and memory bandwidth utilization relative to the peak performance of this device. Achieved compute throughput and/or memory bandwidth below 60.0% of peak typically indicate latency issues. Look at Scheduler Statistics and Warp State Statistics for potential reasons.

Key Performance Indicators

Roofline Analysis The ratio of peak float (FP32) to double (FP64) performance on this device is 32:1. The workload achieved close to 0% of this device's FP32 peak performance and 13% of its FP64 peak performance. See the Roofline chart for more details on roofline analysis.

GPU Throughput



PM Sampling

Timeline view of PM metrics sampled periodically over the workload duration. Data is collected across multiple passes. Use this section to understand how workload behavior changes over its runtime.

Maximum Sampling Interval [cycle]	20000	# Pass Groups	1
Maximum Buffer Size [Mbyte]	2.03		

Compute Workload Analysis

Pipe Utilization (Elapsed Cycles)

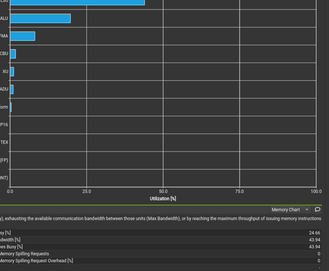
Detailed analysis of the compute resources of the streaming multiprocessors (SM), including the achieved instructions per clock (IPC) and the utilization of each available pipeline. Pipelines with very high utilization might limit the overall performance.

Executed Ipc Elapsed [inst/cycle]	0.96	SM Busy [%]	57.13
Executed Ipc Active [inst/cycle]	0.98	Issue Slots Busy [%]	24.24
Issued Ipc Active [inst/cycle]	0.99		

Balanced FP64 is the highest-utilized pipeline (57.1%) based on elapsed cycles in the workload, taking into account the rates of its different instructions. It executes 64-bit floating point operations. It is well-utilized, but should not be a bottleneck.

Pipe Utilization (% of elapsed cycles)

Pipe Utilization (% of peak instructions executed over elapsed cycles)



Memory Workload Analysis

Memory Chart

Detailed analysis of the memory resources of the GPU. Memory can become a limiting factor for the overall kernel performance when fully utilizing the involved hardware units (Mem Busy), exhausting the available communication bandwidth between those units (Max Bandwidth), or by reaching the maximum throughput of issuing memory instructions (Mem Pipes Busy). Detailed chart of the memory units. Detailed tables with data for each memory unit.

Memory Throughput [Gbyte/s]	42.42	Mem Busy [%]	24.66
L1/TEX Hit Rate [%]	74.40	Max Bandwidth [%]	43.94
L2 Hit Rate [%]	93.98	Mem Pipes Busy [%]	43.94
Local Memory Spilling Requests	0	Shared Memory Spilling Requests	0
Local Memory Spilling Request Overhead [%]	0	Shared Memory Spilling Request Overhead [%]	0
L2 Sector Promotion Misses [%]	16.38		

Local Memory Usage This workload accesses 8 local sectors, accounting for 10.70% of all sectors requested in L1/TEX. Local memory refers to a memory space that is private to each thread. Although it is logically distinct, local memory resides in global memory. This can lead to higher memory latency compared to the usage of registers or shared memory, a decrease in available bandwidth in global memory, and an increase in executed instructions.
Est. Speedup: 22.85% 39.81% of all local loads and 83.37% of all local stores spill into the L2 cache, accounting for 13.66% of all sectors requested in L2.
One common reason for the use of local memory is arrays being too large to fit into registers or not being indexed using compile time constants (see the 'Local Memory' section of the CUDA Programming Guide). For more advice on how to reduce the usage of local memory refer to the Optimization.

Key Performance Indicators

L1/TEX Global Load Access Pattern **Est. Speedup: 40.86%** The memory access pattern for global loads from L1/TEX might not be optimal. On average, only 4.1 of the 32 bytes transmitted per sector are utilized by each thread. This could possibly be caused by a stride between threads. Check the Memory Coalescing section for uncoalesced global loads.

Key Performance Indicators

L1/TEX Global Store Access Pattern **Est. Speedup: 14.51%** The memory access pattern for global stores to L1/TEX might not be optimal. On average, only 22.6 of the 32 bytes transmitted per sector are utilized by each thread. This could possibly be caused by a stride between threads. Check the Memory Coalescing section for uncoalesced global stores.

Key Performance Indicators

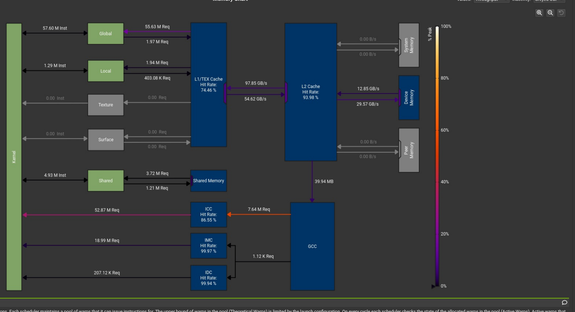
L1/TEX Local Load Access Pattern **Est. Speedup: 36.64%** The memory access pattern for local loads from L1/TEX might not be optimal. On average, only 8.1 of the 32 bytes transmitted per sector are utilized by each thread. This could possibly be caused by a stride between threads. Check the Memory Coalescing section for uncoalesced local loads.

Key Performance Indicators

L1/TEX Local Store Access Pattern **Est. Speedup: 47.68%** The memory access pattern for local stores to L1/TEX might not be optimal. On average, only 1.1 of the 32 bytes transmitted per sector are utilized by each thread. This could possibly be caused by a stride between threads. Check the Memory Coalescing section for uncoalesced local stores.

Key Performance Indicators

Memory Chart



Scheduler Statistics

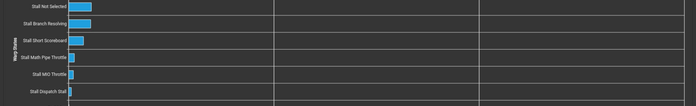
Summary of the activity of the schedulers issuing instructions. Each scheduler maintains a pool of warps that it can issue instructions for. The upper bound of warps in the pool (Theoretical Warps) is limited by the launch configuration. On every cycle each scheduler checks the state of the allocated warps in the pool (Active Warps). Active warps that are not stalled (Eligible Warps) are ready to issue their next instruction. From the set of eligible warps the scheduler selects a single warp from which to issue one or more instructions (Issued Warps). On cycles with no eligible warps, the issue slot is skipped and no instruction is issued. Having many skipped issue slots indicates poor latency hiding.

Active Warps Per Scheduler [warp]	7.40	No Eligible [%]	75.26
Eligible Warps Per Scheduler [warp]	0.38	One or More Eligible [%]	24.80
Issued Warps Per Scheduler	0.25		

Issue Slot Utilization **Est. Local Speedup: 42.87%** Every scheduler is capable of issuing one instruction per cycle, but for this workload each scheduler only issues an instruction every 4.0 cycles. This might leave hardware resources underutilized and may lead to less optimal performance. Out of the maximum of 8 warps per scheduler, this workload allocates an average of 7.40 active warps per scheduler, but only an average of 0.38 warps were eligible per cycle. Eligible warps are the subset of active warps that are ready to issue their next instruction. Every cycle with no eligible warp results in no instruction being issued and the issue slot remains unused. To increase the number of eligible warps, avoid possible load imbalances due to highly different execution durations per warp. Reducing stalls indicated on the Warp Scheduler chart and Kernel Counters sections can help, too.

Key Performance Indicators

Warps Per Scheduler



Warp State Statistics

Analysis of the states in which all warps spent cycles during the kernel execution. The warp states describe a warp's readiness or inability to issue its next instruction. The warp cycles per instruction define the latency between two consecutive instructions. The higher the value, the more warp parallelism is required to hide this latency. For each warp state, the chart shows the average number of cycles spent in that state per issued instruction. Stalls are not always impacting the overall performance nor are they completely avoidable. Only focus on stall reasons if the schedulers fail to issue every cycle. When executing a kernel with mixed library and user code, these metrics show the combined values.

Warp Cycles Per Issued Instruction [cycle]	29.93	Avg. Active Threads Per Warp	29.34
Warp Cycles Per Executed Instruction [cycle]	30.26	Avg. Not Predicated Off Threads Per Warp	23.79

Barrier Stalls **Est. Speedup: 42.8%** On average, each warp of this workload spends 12.8 cycles being stalled waiting for sibling warps at a CTA barrier. A high number of warps waiting at a barrier is commonly caused by diverging code paths before a barrier. This causes some warps to wait a long time until other warps reach the synchronization point. Whenever possible, try to divide up the work into blocks of uniform workloads. If the block size is 512 threads or greater, consider splitting it into smaller groups. This can increase eligible warps without affecting occupancy, unless shared memory becomes a new occupancy limiter. Also, try to identify which barrier instruction causes the most stalls, and optimize the code executed before that synchronization point first. This stall type represents about 43.1% of the total average of 29.8 cycles between issuing two instructions.

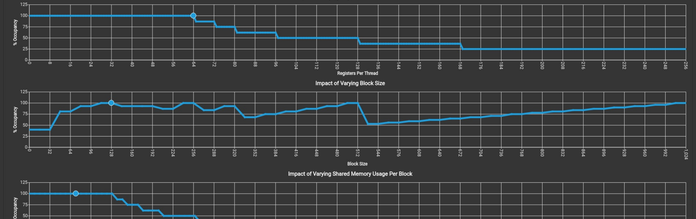
Key Performance Indicators

Warp Stall Check the Warps Scheduler and Instruction Statistics table for the top stall locations in your source based on sampling data. The Profiling Guide provides more details on each stall reason.

Thread Divergence **Est. Speedup: 20.12%** Instructions are executed in warps, which are groups of 32 threads. Optimal instruction throughput is achieved if all 32 threads of a warp execute the same instruction. The chosen launch configuration, early thread completion, and divergent flow control can significantly lower the number of active threads in a warp per cycle. This workload achieves an average of 26.3 threads being active per cycle. This is further reduced to 20.7 threads per warp due to predication. The compiler may use predication to avoid an actual branch. Instead, all instructions are scheduled, but a per-thread condition code or predicate controls which threads execute the instructions. Try to avoid different execution paths within a warp when possible.

Key Performance Indicators

Warp State (All Cycles)



Instruction Statistics

Opcode Category Chart

Statistics of the executed low-level assembly instructions (SASS). The instruction mix provides insight into the types and frequency of the executed instructions. A narrow mix of instruction types implies a dependency on few instruction pipelines, while others remain unused. Using multiple pipelines allows hiding latencies and enables parallel execution. Note that instructions/Opcode and Executed Instructions/ are measured differently and can diverge if cycles are spent in system calls.

Executed Instructions [inst]	303423637	Avg. Executed Instructions Per Scheduler [inst]	2291056.15
Issued Instructions [inst]	307846176	Avg. Issued Instructions Per Scheduler [inst]	2323545.41
Local Memory Spilling Requests [inst]	0	Shared Memory Spilling Requests [inst]	0

FP64 Non-Fused Instructions **Est. Speedup: 10.42%** This workload executes 6702436 fused and 3850091 non-fused FP64 instructions. By converting pairs of non-fused instructions to their DFMA, higher throughput equivalent, the achieved FP64 performance could be increased by up to 18% (relative to its current performance).

Key Performance Indicators

NVLink Topology

NVLink Topology diagram shows logical NVLink connections with transmit/receive throughput.

NVLink Tables

Detailed tables with properties for each NVLink.

NUMA Affinity

Non-uniform memory access (NUMA) affinities based on compute and memory distances for all GPUs.

Launch Statistics

Summary of the configuration used to launch the kernel. The launch configuration defines the size of the kernel grid, the division of the grid into blocks, and the GPU resources needed to execute the kernel. Choosing an efficient launch configuration maximizes device utilization.

Grid Size	977	Cluster Scheduling Policy	CachePreferNone
Registers Per Thread [register/thread]	64	Static Shared Memory Per Block [Kbyte/block]	4.61
Block Size	128	Dynamic Shared Memory Per Block [Byte/block]	0
Threads [thread]	125056	Driver Shared Memory Per Block [Byte/block]	1
Waves Per SM	3.59	Shared Memory Configuration Size [Kbyte]	65.54
Uses Green Context	0	Stack Size	1024
# SMs [SM]	34	# TPCs	17
Enabled TPC IDs	all		

Tail Effect **Est. Speedup: 29.08%** A wave of thread blocks is defined as the maximum number of blocks that can be executed in parallel on the target GPU. The number of blocks in a wave depends on the number of multiprocessors and the theoretical occupancy of the kernel. This kernel launch results in 3 full waves and a partial wave of 161 thread blocks. Under the assumption of a uniform execution duration of all thread blocks, this partial wave may account for up to 25.0% of the total runtime of this kernel. Try launching a grid with no partial wave. The overall impact of this tail effect also lessens with the number of full waves executed for a grid. See the Hardware Model description for more details on launch configurations.

Key Performance Indicators

Occupancy

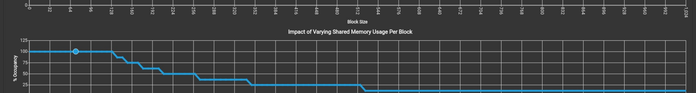
Occupancy Graphs

Occupancy is the ratio of the number of active warps per multiprocessor to the maximum number of possible active warps. Another way to view occupancy is the percentage of the hardware's ability to process warps that is actively in use. Higher occupancy does not always result in higher performance, however, low occupancy always reduces the ability to hide latencies, resulting in overall performance degradation. Large discrepancies between the theoretical and the achieved occupancy during execution typically indicates highly imbalanced workloads.

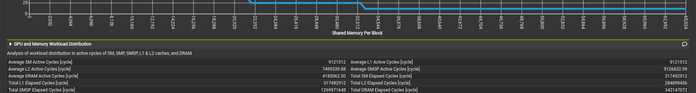
Theoretical Occupancy [%]	100	Block Limit Registers [block]	32
Theoretical Active Warps per SM [warp]	32	Block Limit Shared Mem [block]	13
Achieved Occupancy [%]	92.40	Block Limit Warps [block]	8
Achieved Active Warps Per SM [warp]	29.57	Block Limit SM [block]	16

Impact of Varying Register Count Per Thread

Impact of Varying Block Size



Impact of Varying Shared Memory Usage Per Block



GPU and Memory Workload Distribution

Analysis of workload distribution in active cycles of SM, SMP, SMSP, L1 & L2 caches, and DRAM

Average SM Active Cycles [cycle]	9121512	Average L1 Active Cycles [cycle]	9121512
Average L2 Active Cycles [cycle]	7499339.88	Average SMSP Active Cycles [cycle]	9126832.99
Average DRAM Active Cycles [cycle]	4169043.50	Total SM Elapsed Cycles [cycle]	317485011
Total L1 Elapsed Cycles [cycle]	317409312	Total L2 Elapsed Cycles [cycle]	284849456
Total SMSP Elapsed Cycles [cycle]	1269917648	Total DRAM Elapsed Cycles [cycle]	142147072

Source Counters

Source metrics, including branch efficiency and sampled warp stall reasons. Warp Stall Sampling metrics are periodically sampled over the kernel runtime. They indicate when warps were stalled and couldn't be scheduled. See the documentation for a description of all stall reasons. Only focus on stalls if the schedulers fail to issue every cycle.

Branch Instructions [inst]	29790853	Branch Efficiency [%]	98.30
Branch Instruction Ratio [%]	0.10	Avg. Divergent Branches [branches]	3135.51

Uncoalesced Global Accesses **Est. Speedup: 6.45%** This workload has uncoalesced global accesses resulting in a total of 5831003 excessive sectors (8% of the total 76108865 sectors). Check the L2 Theoretical Sectors Global Excessive table for the primary source locations. The CUDA Programming Guide has additional information on reducing uncoalesced device memory accesses.

Key Performance Indicators

L2 Theoretical Sectors Global Excessive

Location	Value	Value [%]
0x5000f48f70 in circuit_kernel_batch_plain_stack	392,160	
0x5000f48f80 in circuit_kernel_batch_plain_stack	392,160	
0x5000f48fb0 in circuit_kernel_batch_plain_stack	392,160	
0x5000f48fc0 in circuit_kernel_batch_plain_stack	392,160	
0x5000f48f20 in circuit_kernel_batch_plain_stack	392,160	

Follow the rules outputs to get guidance on how to navigate through the report and quickly discover performance bottlenecks in this kernel.
You could also disable Rule Hints & Rules to focus on selected performance aspects and make profiling faster.